ENGINEERING FIELD GUIDE · INTERVIEW PREPARATION

AI Career Navigator

Project structure, frontend and backend walkthrough, MongoDB data model, Groq AI flow, and interview-ready explanations.

A practical guide to understanding the whole application: what each folder owns, how a candidate request travels through the system, and where to make changes safely.

Project: skill-quest-career-navigator
Stack: React 18 · TypeScript · Vite · Express 5 · MongoDB/Mongoose · Groq
Prepared from: the current workspace source and configuration

How to Use This Guide

Read the overview and request-flow sections first. Then use the file catalog as a map while opening source files in the editor. The interview questions at the end are designed to help explain trade-offs in your own words, not to memorize lines verbatim.

> **Scope note.** This document covers every first-party source directory and project configuration file. It explains each feature module and inventories the UI primitives individually. `node_modules/` is installed third-party code and `dist/` is generated build output; neither is hand-maintained application source. Private `.env` values are intentionally not reproduced here.

Contents

1. Project in one minute
2. System architecture and request flow
3. Application routes and candidate journeys
4. Backend API, authentication, and MongoDB document
5. Groq integration and prompt processing
6. Feature-by-feature frontend guide
7. Complete folder and file catalog
8. UI primitives inventory
9. Configuration, running, and validation
10. Security boundaries and known limitations
11. Interview practice questions

1. Project in One Minute

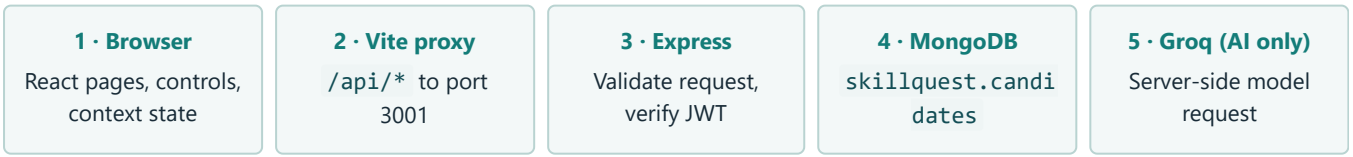
AI Career Navigator is a single-page career coaching application. A candidate creates an account, uploads or pastes resume text, asks Groq to analyze or generate career material, and keeps the resulting profile data in MongoDB. The React frontend owns the user experience; an Express API owns authentication, authorization, database access, and all calls containing the Groq secret.

> **Interview summary:** "This is a React and TypeScript career coaching app backed by an Express API. Mongoose stores one candidate document per account, passwords are bcrypt-hashed, and a seven-day JWT ties every profile and AI request to the current candidate. Vite proxies the browser's `/api` calls to Express. Groq runs server-side and returns structured JSON for resume review, roadmaps, mindmaps, and interview practice."

Technology map

Layer	Technology	Role in this project
UI	React 18, TypeScript, React Router	Pages, reusable components, form state, and client-side navigation.
Design system	Tailwind CSS, Radix UI wrappers, Lucide	Utility styling, accessible controls, icons, and theme tokens.
Development/build	Vite, SWC, npm	Fast local frontend server and production bundle.
API	Node.js 20+, Express 5	REST endpoints, validation, JWT auth, AI proxy.
Persistence	MongoDB, Mongoose	Candidate accounts and account-owned career profile state.
Authentication	bcryptjs, jsonwebtoken	Password hashing and signed bearer tokens.
AI	Groq OpenAI-compatible HTTP API	Model selection, JSON response generation, structured career assistance.
Resume parsing	pdfjs-dist, mammoth	Extract selectable text from PDF and DOCX in the browser.
Mindmap	React Flow	Interactive node-and-edge visualization.

2. Architecture and Request Flow



During development, the browser loads from `http://localhost:8080`. Its relative API requests go to `/api`; the Vite proxy forwards them to `http://127.0.0.1:3001`. This makes browser and API calls same-origin from the frontend's point of view and avoids exposing a second public origin. Express connects to the MongoDB URI from the server environment. AI routes call Groq directly from Node, so the Groq key never enters a JavaScript bundle.

Typical authenticated request

1. The user submits a form in a page component.
2. `apiRequest()` adds JSON headers and the bearer token from browser local storage.

3. Vite forwards the request to Express. The route checks the Mongo connection and verifies the JWT.
4. For profile writes, the server confirms every top-level field is allowed, merges those fields into that candidate's profile, and saves the Mongoose document.
5. The JSON response returns to the page; `AuthContext` updates its in-memory copy so the UI reflects the saved value.

> **Keep the responsibilities clear:** React state is the immediate view state, not the durable database. MongoDB is the durable account data store. The JWT identifies who may access it. Groq generates suggestions but does not store a candidate profile.

3. Routes and Candidate Journeys

URL	Page/component	Access and purpose
`/`	`Home`	Public landing page and personalized roadmap form. Generation requires an account.
`/auth`	`Auth`	Registration and sign-in form. Returns to the route that required authentication when possible.
`/dashboard`	`Dashboard` + `DashboardHome`	Protected account home with live profile-derived progress.
`/dashboard/career-mapping`	`CareerMapping`	Upload/paste resume, ask Groq for ATS-style review, save text and analysis.
`/dashboard/skill-roadmap`	`SkillRoadmap`	Interactive mindmap tab plus skill self-assessment and completion progress.
`/dashboard/ai-interview`	`AIInterview`	Choose HR or technical interview, generate questions, submit/evaluate answers, persist sessions.
`/dashboard/job-matching`	`JobMatching`	Filter curated company career links and external job portals; saves search preferences.
`/dashboard/mentor-connect`	`MentorConnect`	Mentor directory examples, community links, open-source issue links, persisted session requests.
Other paths	`NotFound`	Fallback 404 screen.

Authentication lifecycle

1. **Register:** POST `/api/auth/register` sends display name, email, password. The server validates, normalizes email, hashes the password with bcrypt cost 12, inserts a Candidate, and returns a JWT and profile.
2. **Restore:** on app load, `AuthProvider` reads `skillquest.session` and calls `/api/auth/me`. The server verifies the token and returns the matching user and profile.
3. **Use the app:** `ProtectedDashboard` renders the dashboard only when the context has a user; otherwise it redirects to `/auth`.
4. **Save:** `saveProfile()` sends a PATCH with a partial profile; the API uses the candidate id in the verified token, not a client-supplied account id.
5. **Sign out:** the client removes the token and clears user/profile memory. MongoDB data remains attached to the account for the next sign-in.

4. Backend API and MongoDB Model

Endpoint reference

Method + path	Auth	Behavior
`GET /api/health`	No token; requires DB	Returns `{ status: "ok", database: "skillquest" }` when Mongo is connected.

Method + path	Auth	Behavior
`POST /api/auth/register`	Public	Validate input, hash password, create account, return token/user/profile.
`POST /api/auth/login`	Public	Find normalized email, compare bcrypt hash, issue fresh token.
`GET /api/auth/me`	Bearer JWT	Restore account and profile for the token subject.
`GET /api/candidate/profile`	Bearer JWT	Read only that candidate's profile.
`PATCH /api/candidate/profile`	Bearer JWT	Validate allow-list and merge profile update into the same candidate.
`POST /api/ai/analyze-resume`	Bearer JWT	Analyze resume text; 30-character minimum, 15,000-character maximum.
`POST /api/ai/generate-roadmap`	Bearer JWT	Generate phases, milestones, schedule, and recommendations.
`POST /api/ai/generate-mindmap`	Bearer JWT	Generate and validate node/edge JSON.
`POST /api/ai/generate-interview-questions`	Bearer JWT	Generate role-specific HR or technical questions.
`POST /api/ai/evaluate-answer`	Bearer JWT	Score an answer and return strengths, improvements, and feedback.

Candidate document

Mongoose model `Candidate` maps to MongoDB collection `candidates` in database `skillquest`. The schema has timestamps, a unique lowercased email index, and a password hash configured with `select: false`.

```
{
  _id: ObjectId,
  displayName: String,
  email: String,                // unique, lowercase, trimmed
  passwordHash: String,         // bcrypt; excluded from normal reads
  profile: {                    // candidate-owned feature data
    resumeText, careerAnalysis,
    roadmapData, roadmapRequest,
    mindmap, selectedRole, skillLevels,
    learningProgress, interviews,
    jobPreferences, mentorRequests
  },
  createdAt: Date,
  updatedAt: Date
}
```

Profile field	What it persists
`resumeText`	Extracted or pasted text; the binary upload itself is not stored.
`careerAnalysis`	Groq summary, ATS score/breakdown, skills, paths, keywords, strengths, improvements, rewrite suggestions.
`roadmapData` / `roadmapRequest`	Generated output plus domain, learning goal, and daily hours input.
`mindmap`	Domain/goal and raw node/edge graph so the visualization can be rebuilt after login.

Profile field	What it persists
`selectedRole` / `skillLevels`	Target role and per-skill current/target percentages.
`learningProgress`	Completion booleans keyed by roadmap item title.
`interviews`	Session id/type/role/focus/date, generated questions, candidate answers, evaluations, final score, completion state.
`jobPreferences`	Selected domain and company filter query. There is no live job-feed record.
`mentorRequests`	Mentor names for which the candidate saved a connection request.

The profile is a MongoDB `Mixed` object. The API limits updates to an explicit top-level allow-list and shallow-merges allowed properties. That keeps account ownership enforced, but does not deeply validate every nested value; a production evolution could replace `Mixed` with typed sub-schemas and request validation.

5. Groq AI Integration

AI generation is centralized in `server/index.js`. The frontend never calls Groq directly. Every `/api/ai/*` route first passes the database and JWT middleware, validates required inputs, builds a system/user prompt, calls the shared `groqJson()` helper, and returns structured JSON.

Model and JSON handling

1. Read `GROQ_API_KEY` from the Node server's environment. If absent, return a clear 503 instead of silently fabricating AI output.
2. Fetch the available Groq models once and cache a preferred available model; fall back to `llama-3.3-70b-versatile` if model discovery cannot choose one.
3. Send an OpenAI-compatible `/chat/completions` request with system/user messages, temperature, token limit, and JSON response format.
4. If a model rejects the JSON response-format option with HTTP 400, retry without it.
5. Parse plain JSON or a fenced JSON object, then validate key shapes for mindmaps/questions and apply safe defaults for resume analysis.

AI job	Input essentials	Expected result
Resume analysis	Resume text, capped at 15k characters	ATS feedback, skills, career paths, keywords, resume suggestions.
Roadmap	Domain, learning goal, optional hours/day	Phases, topics/resources, milestones, weekly goals, daily schedule.
Mindmap	Domain and learning goal	Hierarchical `nodes[]` and `edges[]` rendered by React Flow.
Interview questions	Role and HR/technical focus	Questions with category, difficulty, expected points, time limit.
Answer evaluation	Question, candidate answer, expected points	0–100 score, strengths, improvements, feedback, covered/missed points.

> **Environment separation:** `.env.example` is a safe template. `.env` contains machine-local credentials, is ignored by Git, and must never be imported into client code or included in the PDF. Rotate any key that has been shared or committed.

6. Frontend Features: Code Walkthrough

Home — `src/pages/Home.tsx`

Public entry page with navigation, feature summaries, and a roadmap form. Domain, learning goal, and available daily hours go to `/api/ai/generate-roadmap`. The generated response and original request are saved in the signed-in profile. A visitor can browse the page, but generation redirects to account sign-in.

Career Mapping — `src/pages/CareerMapping.tsx`

Accepts TXT, PDF, and DOCX files or pasted text. PDF extraction uses PDF.js in the browser; DOCX extraction uses Mammoth. Extracted text is trimmed and limited to 15,000 characters, then saved. The analysis request is authenticated and its JSON result is saved for reload. The result view displays ATS score/breakdown, keywords,

rewrite suggestions, experience level, skills, strengths, improvements, and suggested career tracks. The original DOC format is explicitly unsupported.

Skill Roadmap — `src/pages/SkillRoadmap.tsx`

Two tabs: Interactive Mindmap and Skill Assessment. Eight predefined technical roles provide baseline skill targets. Candidate current proficiency is editable by sliders; releasing a slider persists the changed role-specific array. Learning item completion is stored in `learningProgress`. Links/resources are a bundled catalog, not AI-generated database records.

Mindmap — `src/components/MindmapGenerator.tsx`

Prompts for domain and goal; POSTs to Groq through the API; maps saved graph data into React Flow `Node` and `Edge` objects. Node color/spacing is based on hierarchy level. Raw graph plus inputs are saved, then reconstructed when the component mounts. The current Download icon displays an informational toast; actual image/PDF export is not implemented.

AI Interview — `src/pages/AIInterview.tsx`

Flow is select type → setup role/focus → generate questions → answer/evaluate → completion. A UUID identifies a session. Each evaluated answer and the updated question/answer arrays are persisted. Incomplete sessions can be restored from profile state; completed sessions appear in history with average score. HR/technical prompt focus controls the type of Groq questions.

Job Matching — `src/pages/JobMatching.tsx`

Stores selected domain and company filter in the profile. It filters a curated in-code list of company career pages and links to external job boards. It does not query a job API, scrape live listings, submit applications, or calculate a candidate/job match score. The screen says so rather than presenting mock matches as real.

Mentor Connect — `src/pages/MentorConnect.tsx`

Shows a sample mentor catalog, external developer communities, and links to GitHub “good first issue” searches. Clicking a mentor request stores the mentor name on the candidate profile and disables that request button. Community membership and mentor availability are not verified by this app.

Dashboard — `src/pages/Dashboard.tsx` and `DashboardHome.tsx`

Shared shell combines sidebar, nested route outlet, candidate display name, and sign-out. Dashboard home quick actions link to the feature pages. Progress is calculated from profile completion, saved skill values, and completed interview scores, not fixed percentages.

7. Complete Project Tree

```
skill-quest-career-navigator/
├── .env                          private local settings (ignored by Git)
├── .env.example                  safe environment-variable template
├── .gitignore                    ignores secrets, dependencies, build output
├── README.md                     setup and local run instructions
├── package.json                  scripts and runtime/dev dependencies
├── package-lock.json             npm dependency lock
├── components.json               shadcn/ui generator aliases/settings
├── eslint.config.js              ESLint flat configuration
├── index.html                    Vite HTML entry and page metadata
├── postcss.config.js             Tailwind and Autoprefixer plugins
├── tailwind.config.ts            theme tokens, colors, animations
├── tsconfig.json                 shared TypeScript project references/aliases
├── tsconfig.app.json             browser TypeScript options
├── tsconfig.node.json            Vite config TypeScript options
├── vite.config.ts                Vite React plugin, alias, API proxy
├── public/
│   ├── placeholder.svg           generic placeholder asset
│   └── robots.txt                crawler access rules
├── server/
│   └── index.js                  Express API, Mongo model, auth, Groq routes
└── src/
    ├── main.tsx                  React root and CSS entry
    ├── App.tsx                   providers, router, protected dashboard
    ├── App.css                   legacy starter styles; not imported by main
    ├── index.css                 global Tailwind layers and theme variables
    ├── vite-env.d.ts             Vite client type declarations
    ├── contexts/
    │   └── AuthContext.tsx        signed-in candidate and profile state
    ├── hooks/
    │   ├── use-mobile.tsx        responsive breakpoint hook
    │   └── use-toast.ts          toast hook re-export
    ├── lib/
    │   ├── api.ts                API client and candidate data types
    │   └── utils.ts              class-name merge helper
    ├── components/
    │   ├── Navbar.tsx            public navigation and account actions
    │   ├── DashboardSidebar.tsx  feature navigation
    │   ├── MindmapGenerator.tsx  AI mindmap UI/React Flow graph
    │   └── ui/                   reusable Radix/shadcn-style primitives
    └── pages/
        ├── Home.tsx              public landing and roadmap generator
        ├── Auth.tsx              sign-in and registration
        ├── Dashboard.tsx         authenticated dashboard shell
        ├── DashboardHome.tsx     metrics and quick actions
        ├── CareerMapping.tsx     resume import and AI analysis
        ├── SkillRoadmap.tsx      skill scoring and learning progress
        ├── AIInterview.tsx       question/answer practice
        ├── JobMatching.tsx       company and job-board links
        ├── MentorConnect.tsx     mentor/community/project directory
        └── NotFound.tsx          unknown-route fallback
```

Removed integrations: there is no Supabase or builder-specific runtime directory in the current source tree. The project's backend is the local Express service.

8. Folder and File Responsibilities

Path	Contents and why it exists
`src/pages/`	Route-level user experiences. Pages compose shared controls, call typed API functions, and access candidate data through the auth context.
`src/components/`	Reusable app components shared across pages: navigation and mindmap UI.
`src/components/ui/`	Small accessible UI primitives based largely on Radix UI. They provide consistent buttons, menus, dialog, form controls, layout, feedback, and visualization wrappers.
`src/contexts/`	Cross-route state. AuthProvider restores the signed-in user and Mongo-backed profile and exposes login/register/logout/save methods.
`src/hooks/`	Reusable React hooks for responsive viewport behavior and toast notifications.
`src/lib/`	Shared non-visual logic: API transport, domain interfaces, token storage, and Tailwind class merging.
`server/`	Node-only trust boundary. Credentials stay here; this layer owns MongoDB and talks to Groq.
`public/`	Static assets copied to the built site unchanged; robots policy and placeholder graphic.
`dist/`	Generated production site from `npm run build`. Recreate; do not hand-edit.
`node_modules/`	Installed dependencies from npm. Recreate using the package manifest/lockfile.
`docs/`	This project/interview guide and its generated PDF.

Top-level configuration files

File	What to know
`package.json`	Scripts: `dev` runs API and Vite together; `dev:api` watches the Node server; `dev:web` runs Vite; `start` starts API; `build` builds UI; `lint` runs ESLint.
`package-lock.json`	Exact npm dependency tree for repeatable installation. Commit the lock with dependency changes.
`.env.example`	Documents Mongo URI, API port, JWT secret, Groq key variable names without real credentials.
`.gitignore`	Ignores `.env` / `.env.*` (except example), `node_modules`, `dist`, logs, and editor files.
`vite.config.ts`	Runs the React SWC plugin, aliases `@` to `src`, proxies `/api` to port 3001.
`index.html`	Browser document shell. Loads `src/main.tsx` into `#root`.
`tailwind.config.ts`	Class scanning, dark mode, app colors/radii/gradients/shadows/animations, animate plugin.
`src/index.css`	HSL CSS variables and Tailwind base rules. Change design tokens here.

File	What to know
`tsconfig*.json`	Separate app and Vite config compiler scopes; `@/*` resolves to `src/*`.
`eslint.config.js`	TypeScript/React Hooks lint rules; generated UI has a few known lint findings.
`postcss.config.js`	Runs Tailwind and Autoprefixer during CSS processing.
`components.json`	shadcn-style component generator config and path aliases.
`README.md`	Short first-run instructions; this guide is the deeper learning resource.

9. UI Primitive Inventory

These modules are reusable building blocks, not individual product features. Most wrap Radix UI behavior and apply the application's Tailwind classes. Read the page using a primitive before modifying the primitive itself.

File	Responsibility	File	Responsibility
`accordion.tsx`	Expandable stacked sections.	`alert-dialog.tsx`	Modal confirmation requiring an explicit action.
`alert.tsx`	Inline message/status panel.	`aspect-ratio.tsx`	Maintains a chosen media/content aspect ratio.
`avatar.tsx`	Image/avatar with fallback initials.	`badge.tsx`	Compact status/category label.
`breadcrumb.tsx`	Hierarchical navigation path.	`button.tsx`	Shared button variants and sizes.
`calendar.tsx`	Date picker calendar surface.	`card.tsx`	Card, header, title, content, footer composition.
`carousel.tsx`	Previous/next content carousel.	`chart.tsx`	Chart styling/container helpers for Recharts.
`checkbox.tsx`	Accessible binary selection.	`collapsible.tsx`	Show/hide a related content block.
`command.tsx`	Command palette/searchable action list.	`context-menu.tsx`	Right-click action menu.
`dialog.tsx`	Accessible modal dialog.	`drawer.tsx`	Mobile-oriented slide-in panel.
`dropdown-menu.tsx`	Button-triggered action/options menu.	`form.tsx`	React Hook Form integration, labels/errors.
`hover-card.tsx`	Content panel shown on hover/focus.	`input-otp.tsx`	One-time passcode input slots.
`input.tsx`	Styled text/number/email input.	`label.tsx`	Accessible control label.
`menubar.tsx`	Application-style horizontal menus.	`navigation-menu.tsx`	Accessible site navigation primitives.
`pagination.tsx`	Page navigation controls.	`popover.tsx`	Anchored floating content.
`progress.tsx`	Progress bar display.	`radio-group.tsx`	Mutually exclusive selection.
`resizable.tsx`	Resizable pane layout primitives.	`scroll-area.tsx`	Styled cross-platform scroll container.
`select.tsx`	Accessible option selector.	`separator.tsx`	Visual divider.
`sheet.tsx`	Side-panel dialog shell.	`sidebar.tsx`	Responsive collapsible app sidebar.

File	Responsibility	File	Responsibility
`skeleton.tsx`	Loading placeholder shape.	`slider.tsx`	Numeric range control for self-assessed skill.
`sonner.tsx`	Sonner toast provider wrapper.	`switch.tsx`	Accessible on/off toggle.
`table.tsx`	Consistent table markup/styling.	`tabs.tsx`	Tabbed content navigation.
`textarea.tsx`	Styled multi-line text input.	`toast.tsx`	Toast viewport and message primitives.
`toaster.tsx`	Renders queued toast messages.	`toggle-group.tsx`	Grouped toggle controls.
`toggle.tsx`	Single pressed/unpressed control.	`tooltip.tsx`	Short hover/focus help text and provider.
`use-toast.ts`	Toast queue/reducer and notification hook.	—	Toast consumers use the hook through `src/hooks/use-toast.ts`.

10. Configuration and Local Run

Prerequisites

- Node.js 20 or later and npm.
- MongoDB service listening at `mongodb://localhost:27017/`.
- A valid Groq API key in server-only `.env` for AI routes.

First run

```
npm install
# Create .env from .env.example, then set a private GROQ_API_KEY
npm run dev
```

Vite is available at `http://localhost:8080`. Express is at `http://localhost:3001`. The default database is `skillquest`; Mongo creates it and the collection on first write. Check `GET http://localhost:3001/api/health`.

Scripts in `package.json`

Command	Meaning
`npm run dev`	Concurrently starts API and Vite; stops the pair together.
`npm run dev:api`	Starts Node with watch mode for backend changes.
`npm run dev:web`	Starts only the frontend server.
`npm start`	Starts the API process; it expects a production frontend to be hosted separately.
`npm run build`	Generates the static frontend under `dist/`.
`npm run lint`	Runs ESLint across the project.

> **Verified project state:** the workspace build passed; changed application files passed targeted lint; the full lint command has pre-existing errors in `src/components/ui/command.tsx`, `src/components/ui/textarea.tsx`, and `tailwind.config.ts`. A live Mongo profile round-trip/account isolation check passed, and a real authenticated Groq roadmap request returned structured phases and schedule.

11. Security and Design Boundaries

Boundary	Current implementation	Interview explanation / next improvement
Password	bcrypt hash with cost 12; password hash excluded by default in Mongoose.	Never store or return plaintext passwords. Add reset and verification flows for production.
Session	JWT signed with server secret, subject is Mongo user id, expires in 7 days; browser stores it in localStorage.	localStorage is simple but exposed to successful XSS. Production could use secure HttpOnly SameSite cookies, refresh/revocation strategy, and CSP.
Authorization	Every profile/AI route verifies bearer token and queries by its `sub`.	Do not accept a candidate id from the request body as authority. Add tests for cross-account access.
Secrets	Groq key and JWT secret are server environment variables; `.env` ignored.	Rotate keys if exposed; use a secrets manager outside local dev.
Validation	Register and select API fields have manual validation; request JSON limited to 1MB.	Use a schema validator such as Zod on the server, nested profile schemas, rate limiting, and consistent errors.
Resume	File text is extracted in the browser, clipped to 15k characters, text saved in Mongo and sent to Groq.	Resume content is personally identifiable data. Add explicit retention/deletion controls, encryption/storage policy, and consent.
Job/mentor content	Company links, community URLs, and mentor examples are static UI data.	They are not live verification, job ranking, application tracking, or real mentor availability.
Mongo profile	Mixed schema with allow-listed top-level fields.	Typed subdocuments and field-level limits avoid malformed/oversized nested data.

Important vocabulary

`JWT` signed claims used as a bearer session token. `Mongoose` Node object/document mapper for MongoDB. `Middleware` Express function that runs before the route handler. `Allow-list` fields the API explicitly permits a profile update to change. `Proxy` Vite forwards a browser path to a separate backend during local development. `JSON mode` model request option encouraging parseable structured output.

12. Interview Practice

Answer with a clear sequence: problem → design decision → data/request flow → trade-off. Tie each explanation to a real file or endpoint.

Q1. Walk me through the architecture.

React/TypeScript pages are served by Vite. The browser calls relative `/api` endpoints; Vite proxies them to Express. Express validates and authenticates requests, reads/writes Mongo through Mongoose, and calls Groq for AI routes. Groq and Mongo credentials stay on the server.

Q2. How does a candidate's profile remain private and load after login?

Login returns a JWT whose subject is the Mongo candidate id. The client stores the token and sends it as a bearer header. Middleware verifies it; profile routes query only `req.candidateId`. On reload, the auth provider calls `/api/auth/me` and hydrates the saved profile. Sign-out deletes client session state but not the database record.

Q3. How are passwords handled?

The server hashes at registration with bcrypt cost 12. The password hash is excluded in ordinary Mongoose selects. At login, bcrypt compares the supplied password with the stored hash. A JWT is returned only after successful comparison.

Q4. Why not call Groq from the React app?

A browser bundle is public, so any embedded API key can be copied and abused. The frontend calls an authenticated local API; only the Express process attaches the Groq authorization header.

Q5. What happens if MongoDB is down?

Startup attempts to connect with a short selection timeout and exits if it cannot connect. Routes also check Mongoose connection state and return 503 when the database is unavailable. The health endpoint is the quick operational check.

Q6. What does the API do with model output?

It asks for JSON, retries without the response-format option for models that reject it, parses fenced or plain JSON, and checks required arrays for mindmaps and interview questions. Resume analysis gets safe defaults for missing fields.

Q7. What gets stored for a resume upload?

The browser extracts selectable text from TXT/PDF/DOCX, truncates it at 15k characters, then persists that text and the generated analysis in the candidate profile. The uploaded binary is not sent to Mongo.

Q8. Does “Job Matching” currently match actual jobs?

No live job source is connected. It stores the user’s domain/filter and displays curated employer career pages and job boards. It does not claim fabricated fit scores or application counts.

Q9. How are interview attempts saved?

A session receives a UUID and stores role/type/focus/date, questions, answers, evaluation feedback, completion state, and final average. The session is updated after each evaluated answer so progress survives a logout or reload.

Q10. What are the most important production improvements?

Use secure HttpOnly cookies or stronger token lifecycle controls, add rate limits and schema validation, add automated API/auth/data-isolation tests, model profile fields with typed subdocuments, build resume deletion/retention controls, and connect a real job feed before advertising matching results.

Q11. How would you add a new candidate profile field?

Define its TypeScript interface in `src/lib/api.ts`, add it to the server profile allow-list, persist/load it through `saveProfile()`, and write a focused test proving it saves for one account and cannot cross account boundaries.

Q12. Why use an Express API instead of connecting React directly to MongoDB?

Database credentials and authorization logic cannot safely live in a browser. The API is the trust boundary: it controls secrets, validates inputs, verifies identity, and scopes queries to the authenticated candidate.

13. Source Reading Order

1. `src/main.tsx` — find the React mount point.

2. `src/App.tsx` — understand providers and route protection.
3. `src/contexts/AuthContext.tsx` and `src/lib/api.ts` — understand account/profile state and request headers.
4. `server/index.js` — inspect the server trust boundary, Mongo model, routes, and Groq helper.
5. `src/pages/Auth.tsx` — follow register/login UI.
6. `src/pages/CareerMapping.tsx` — follow file parsing, AI request, and persistence.
7. `src/pages/AIInterview.tsx` — follow a multi-step stateful workflow.
8. `src/pages/SkillRoadmap.tsx` and `src/components/MindmapGenerator.tsx` — follow stored skills and graph rendering.
9. Read Job Matching and Mentor Connect for the distinction between static catalogs and saved candidate preferences.
10. Finish with `vite.config.ts`, `package.json`, and CSS tokens to understand how it runs and looks.

> **One-line mental model:** pages collect intent; `api.ts` attaches session identity; Express enforces identity and validation; Mongo stores candidate-owned state; Groq supplies generated suggestions; the context updates the current screen after durable writes.

End of guide · Use this as a map back to source, then explain the reasoning in your own words.